

Assignment #9: Huffman, BST & Heap

Updated 1834 GMT+8 Apr 15, 2025

2025 spring, Compiled by 李皓翔 物院

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

LC222.完全二叉树的节点个数

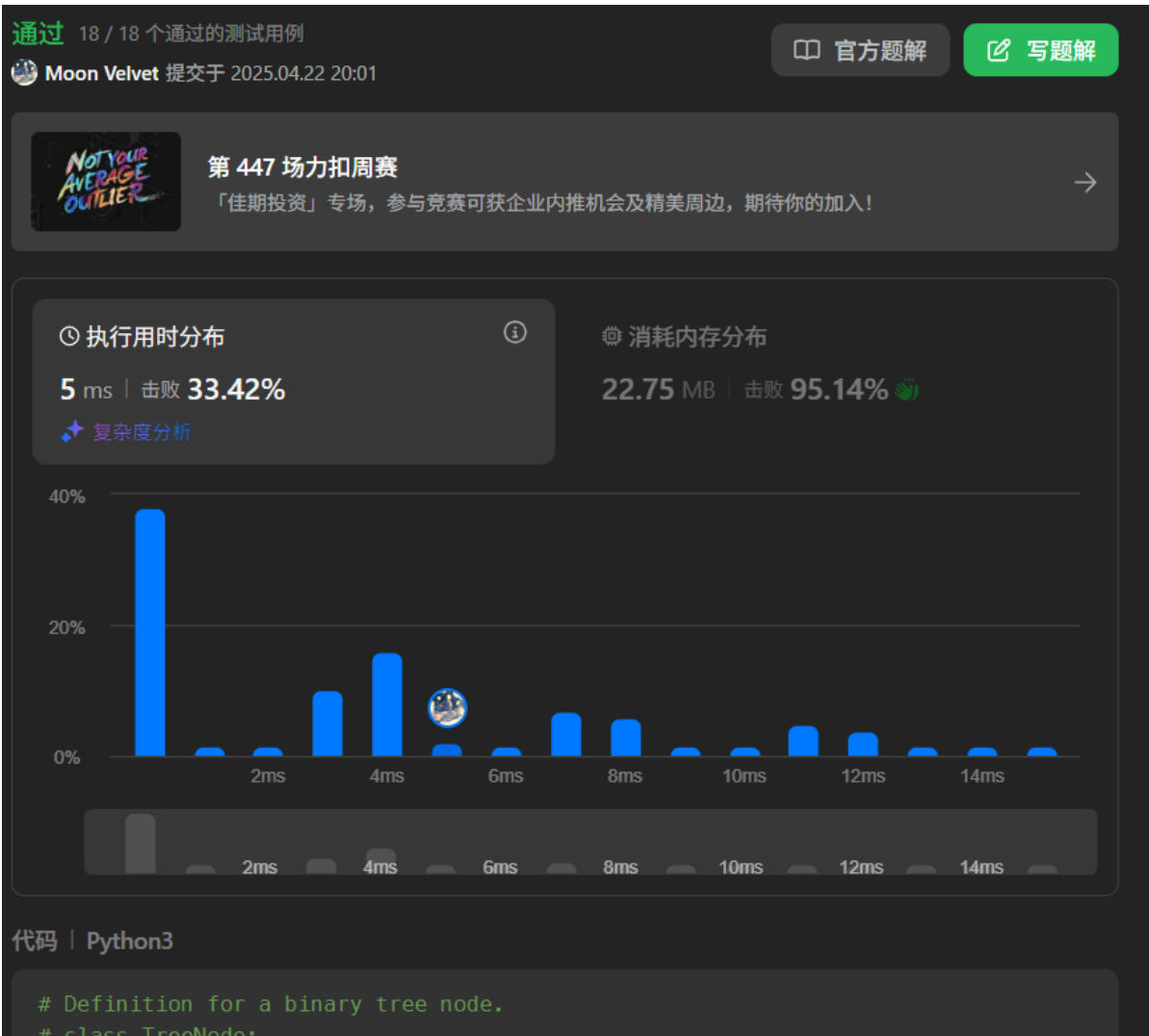
dfs, <https://leetcode.cn/problems/count-complete-tree-nodes/>

思路:

代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countNodes(self, root: Optional[TreeNode]) -> int:
        if root==None:
            return 0
        return 1+self.countNodes(root.left)+self.countNodes(root.right)
```

代码运行截图 (至少包含有"Accepted")



LC103.二叉树的锯齿形层序遍历

bfs, <https://leetcode.cn/problems/binary-tree-zigzag-level-order-traversal/>

思路：

代码：

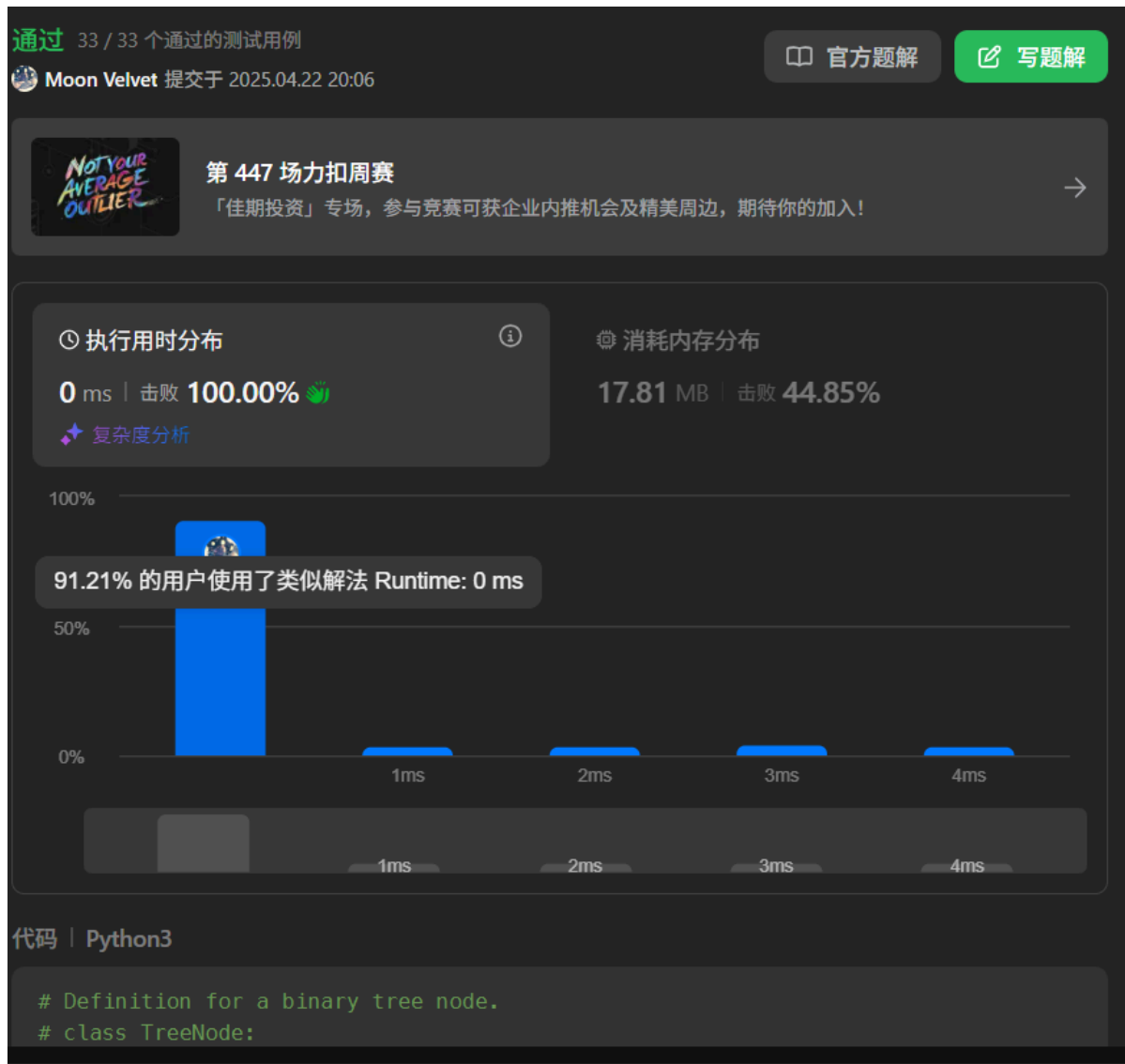
```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        ans, deque = [], collections.deque([root])
        while deque:
            tmp = collections.deque()
            for _ in range(len(deque)):
                node = deque.popleft()
```

```

if len(ans) % 2 == 0:
    tmp.append(node.val)
else:
    tmp.appendleft(node.val)
if node.left:
    deque.append(node.left)
if node.right:
    deque.append(node.right)
ans.append(list(tmp))
return ans

```

代码运行截图 (至少包含有"Accepted")



M04080:Huffman编码树

greedy, <http://cs101.openjudge.cn/practice/04080/>

思路:

代码:

```

from heapq import *

n=int(input())
h=list(map(int,input().split()))
heapify(h)
ans=0
for _ in range(n-1):
    a=heappop(h)
    b=heappop(h)
    heappush(h,a+b)
    ans+=a+b
print(ans)

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```

from heapq import *

n=int(input())
h=list(map(int,input().split()))
heapify(h)
ans=0
for _ in range(n-1):
    a=heappop(h)
    b=heappop(h)
    heappush(h,a+b)
    ans+=a+b
print(ans)

```

M05455: 二叉搜索树的层次遍历

<http://cs101.openjudge.cn/practice/05455/>

思路:

代码:

```

from collections import deque

class TreeNode:
    def __init__(self,value,right=None,left=None):
        self.value=value
        self.right=right
        self.left=left

```

```

def insert(root,value):
    if root.value==value:
        return 0
    if root.value<value:
        if root.right:
            insert(root.right,value)
        else:
            root.right=TreeNode(value)
    else:
        if root.left:
            insert(root.left,value)
        else:
            root.left=TreeNode(value)

def travel(root):
    ans=[]
    d=deque([root])
    while d:
        cur=d.popleft()
        if cur.left:
            d.append(cur.left)
        if cur.right:
            d.append(cur.right)
        ans.append(str(cur.value))
    return ans

l=list(map(int,input().split()))
root=TreeNode(l[0])
for i in range(1,len(l)):
    insert(root,l[i])
print(' '.join(travel(root)))

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

基本信息

#: 48986952

题目: 05455

提交人: 24n24000:

内存: 3648kB

时间: 21ms

语言: Python3

提交时间: 2025-04-2

源代码

```
from collections import deque

class TreeNode:
    def __init__(self, value, right=None, left=None):
        self.value=value
        self.right=right
        self.left=left

def insert(root, value):
    if root.value==value:
        return 0
    if root.value<value:
        if root.right:
            insert(root.right, value)
        else:
            root.right=TreeNode(value)
    else:
        if root.left:
            insert(root.left, value)
        else:
            root.left=TreeNode(value)

def travel(root):
    ans=[]
    d=deque([root])
    while d:
        cur=d.popleft()
        if cur.left:
            d.append(cur.left)
        if cur.right:
            d.append(cur.right)
        ans.append(str(cur.value))
    return ans

l=list(map(int, input().split()))
```

M04078: 实现堆结构

手搓实现, <http://cs101.openjudge.cn/practice/04078/>

类似的题目是 晴问9.7: 向下调整构建大顶堆, <https://sunnywhy.com/sfbj/9/7>

思路:

代码:

```
from collections import deque

class TreeNode:
    def __init__(self, value, right=None, left=None):
        self.value=value
        self.right=right
        self.left=left

def heapadd(root, value):
    if root.value<=value:
        if root.right:
            heapadd(root.right, value)
        else:
```

```

        root.right=TreeNode(value)
    else:
        if root.left:
            heapadd(root.left,value)
        else:
            root.left=TreeNode(value)

def heappop(root):
    pre=root
    while root.left:
        pre=root
        root=root.left
    pre.left=root.right
    return root.value

root=TreeNode(float('inf'))
n=int(input())
for i in range(n):
    tem=list(map(int,input().split()))
    if len(tem)==2:
        heapadd(root,tem[1])
    else:
        print(heappop(root))

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

基本信息

源代码

#

题目

提交人

内存

时间

语言

提交时间

```
from collections import deque

class TreeNode:
    def __init__(self, value, right=None, left=None):
        self.value=value
        self.right=right
        self.left=left

def heapadd(root, value):
    if root.value<=value:
        if root.right:
            heapadd(root.right, value)
        else:
            root.right=TreeNode(value)
    else:
        if root.left:
            heapadd(root.left, value)
        else:
            root.left=TreeNode(value)

def heappop(root):
    pre=root
    while root.left:
        pre=root
        root=root.left
    pre.left=root.right
    return root.value

root=TreeNode(float('inf'))
n=int(input())
for i in range(n):
    tem=list(map(int, input().split()))
    if len(tem)==2:
        heapadd(root, tem[1])
    else:
        print(heappop(root))
```

T22161: 哈夫曼编码树

greedy, <http://cs101.openjudge.cn/practice/22161/>

思路:

代码:

```
import heapq

class Node:
    def __init__(self, chars, weight, left=None, right=None):
        self.chars = chars
        self.weight = weight
        self.left = left
        self.right = right
```

```

def __lt__(self, other):
    if self.weight < other.weight:
        return True
    elif self.weight == other.weight:
        return min(self.chars) < min(other.chars)
    else:
        return False

def build_huffman_tree(frequencies):
    heap = []
    for char, weight in frequencies.items():
        node = Node({char}, weight)
        heapq.heappush(heap, node)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged_chars = left.chars.union(right.chars)
        merged_weight = left.weight + right.weight
        parent = Node(merged_chars, merged_weight, left, right)
        heapq.heappush(heap, parent)

    return heapq.heappop(heap)

def build_codebook(root):
    codebook = {}
    def traverse(node, code):
        if len(node.chars) == 1:
            char = next(iter(node.chars))
            codebook[char] = code
            return
        if node.left:
            traverse(node.left, code + '0')
        if node.right:
            traverse(node.right, code + '1')
    traverse(root, '')
    return codebook

def build_decoding_map(root):
    decoding_map = {}
    stack = [(root, '')]
    while stack:
        node, code = stack.pop()
        if len(node.chars) == 1:
            char = next(iter(node.chars))
            decoding_map[code] = char
        if node.left:
            stack.append((node.left, code + '0'))
        if node.right:
            stack.append((node.right, code + '1'))
    return decoding_map

def encode_string(s, codebook):
    encoded = []
    for char in s:
        encoded.append(codebook[char])

```

```

        return ''.join(encoded)

def decode_string(s, root):
    decoded = []
    current_node = root
    for bit in s:
        if bit == '0':
            current_node = current_node.left
        else:
            current_node = current_node.right
        if len(current_node.chars) == 1:
            char = next(iter(current_node.chars))
            decoded.append(char)
            current_node = root
    return ''.join(decoded)

def main():
    n = int(input())
    frequencies = {}
    for _ in range(n):
        parts = input().split()
        char = parts[0]
        weight = int(parts[1])
        frequencies[char] = weight

    root = build_huffman_tree(frequencies)
    codebook = build_codebook(root)
    decoding_map = build_decoding_map(root)

    while True:
        try:
            line = input().strip()
            if not line:
                continue
            if line[0] in ('0', '1'):
                print(decode_string(line, root))
            else:
                encoded = encode_string(line, codebook)
                print(encoded)
        except EOFError:
            break

main()

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```
import heapq

class Node:
    def __init__(self, chars, weight, left=None, right=None):
        self.chars = chars
        self.weight = weight
        self.left = left
        self.right = right

    def __lt__(self, other):
        if self.weight < other.weight:
            return True
        elif self.weight == other.weight:
            return min(self.chars) < min(other.chars)
        else:
            return False

def build_huffman_tree(frequencies):
    heap = []
    for char, weight in frequencies.items():
        node = Node({char}, weight)
        heapq.heappush(heap, node)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged_chars = left.chars + right.chars
```

基本信息

#: 48987377
题目: 22161
提交人: 24n2400011450
内存: 3804kB
时间: 23ms
语言: Python3
提交时间: 2025-04-22 21:

2. 学习总结和收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。